

Documentation
Assignment 1
Programming Technology

1. Description Of The Task

Simulate a simplified Capital game. There are some players with different strategies, and a cyclical board with several fields. Players can move around the board, by moving forward with the amount they rolled with a dice. A field can be a property, service, or lucky field. A property can be bought for 1000, and stepping on it the next time the player can build a house on it for 4000. If a player steps on a property field which is owned by somebody else, the player should pay to the owner 500, if there is no house on the field, or 2000, if there is a house on it. Stepping on a service field, the player should pay to the bank (the amount of money is a parameter of the field). Stepping on a lucky field, the player gets some money (the amount is defined as a parameter of the field). There are three different kind of strategies exist. Initially, every player has 10000. Greedy player: If he steps on an unowned property, or his own property without a house, he starts buying it, if he has enough money for it. Careful player: he buys in a round only for at most half the amount of his money. Tactical player: he skips each second chance when he could buy. If a player has to pay, but he runs out of money because of this, he loses. In this case, his properties are lost, and become free to buy.

2. Description Of Each Method

2.1. Class Field (Abstract)

`stepOn(Player player)`: Abstract method that defines the trigger behavior when a player lands on this field. Subclasses must implement their own concrete interaction.

2.2. Class PropertyField (extends Class Field)

`stepOn(Player player)`: Handles field landing interactions: offers land for purchase if unowned, offers house construction if landed on by its owner, or charges rent and transfers funds if landed on by an opponent.

`reset()`: Clears field ownership and removes any built house, returning the property to its initial unowned state when a player goes bankrupt.

`buy(Player player)`: Assigns ownership of this property field to the designated player.

`buildHouse()`: Sets the house flag to true, indicating a house has been built on this property.

2.3. Class ServiceField (extends Class Field)

`stepOn(Player player)`: Forces the player who landed on this field to pay a fixed service fee directly to the bank.

2.4. Class LuckyField (extends Class Field)

`stepOn(Player player)`: Rewards the player who landed on this field with a fixed cash bonus added directly to their balance.

2.5. Class Player (Abstract)

`move(int steps, int boardSize)`: Updates the player's position clockwise along the board using modulo arithmetic (%) to maintain a circular track.

`increaseBalance(int amount)`: Adds the specified amount to the player's current balance.

`decreaseBalance(int amount)`: Subtracts the specified amount from the player's balance; triggers bankruptcy if the balance falls below zero.

`addProperty(PropertyField property)`: Registers a newly acquired property into the player's list of owned assets.

`bankrupt()`: Marks the player as bankrupt, iterates through all owned properties to call `reset()`, and clears the property list.

`shouldBuy(int cost)`: Abstract decision method that evaluates whether the player chooses to spend `cost` on purchasing property or building a house according to their specific strategy.

2.6. Class GreedyPlayer (extends Class Player)

`shouldBuy(int cost)`: Implements the greedy strategy: returns `true` to purchase whenever the current balance is greater than or equal to the cost.

2.7. Class CarefulPlayer (extends Class Player)

`shouldBuy(int cost)`: Implements the careful strategy: returns `true` only if the cost does not exceed half of the player's current balance.

2.8. Class TacticalPlayer (extends Class Player)

`shouldBuy(int cost)`: Implements the tactical strategy: alternates a toggle flag on every purchase opportunity to skip every second chance, returning `true` only on active turns if balance is sufficient.